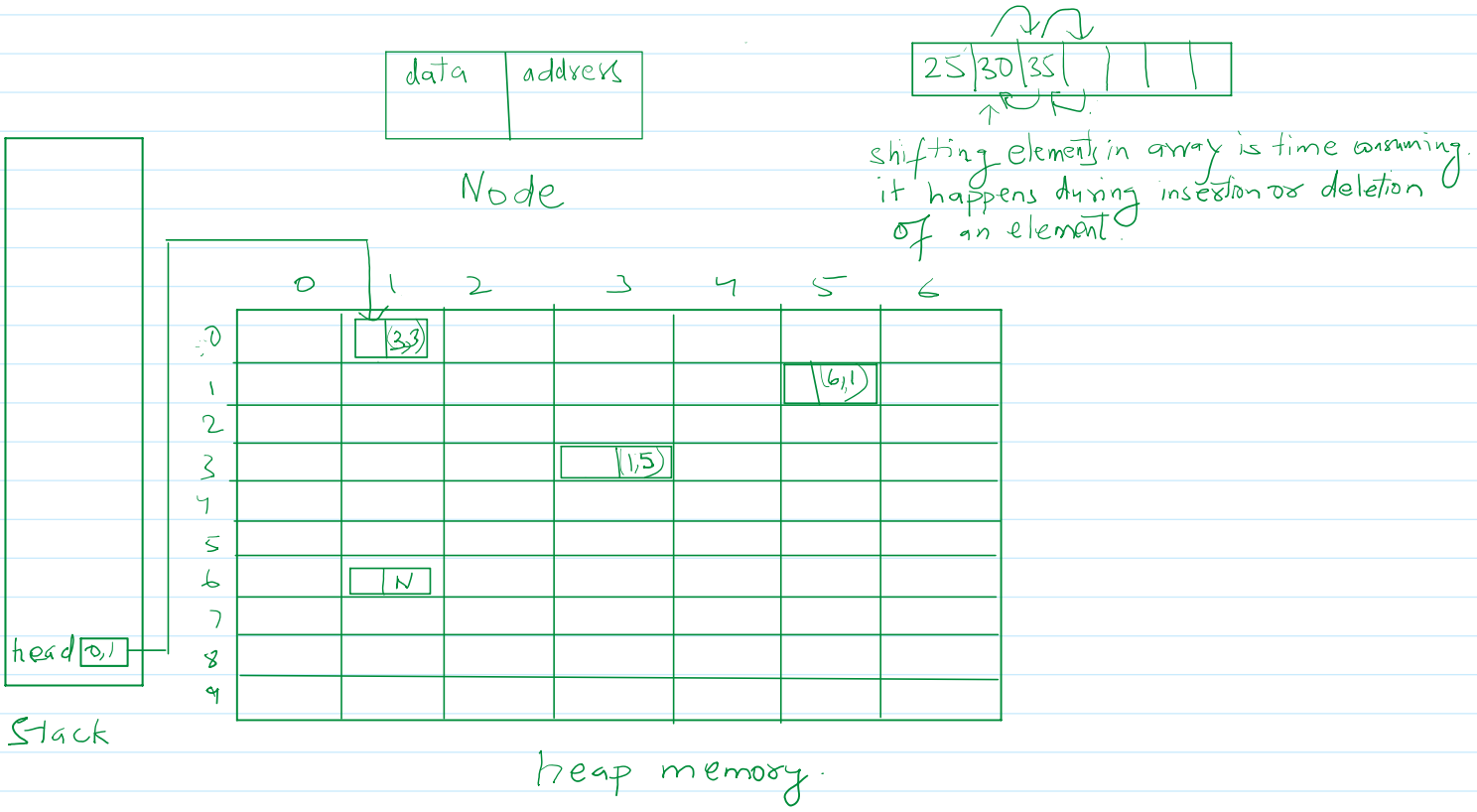
Linked List

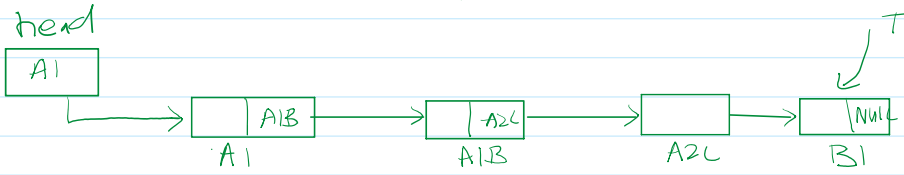
02 September 2026 20:09

A linked list is a linear data structure that includes series of connected nodes. A linguist can be defined as a set of nodes that are randomly stored in memory. A node in the linked list contains 2 parts. That is the first is the data part and the 2nd is the address part. The last node of the list contains a pointer to NULL. After array Linked list is the second most used data structure. In a linked list, every link contains a connection to another link.



What is a Node?

A node is a self referential structure / class. It means, it can hold the reference of another node in it of its own type.



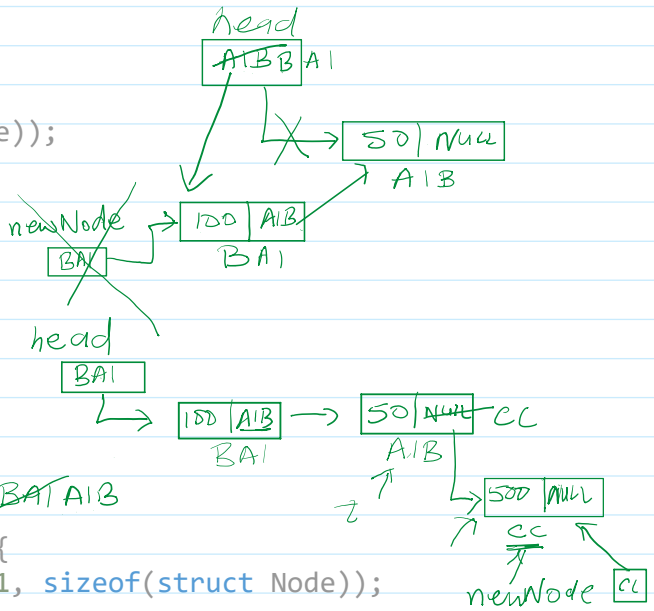
Linked List

How to declare a Node?

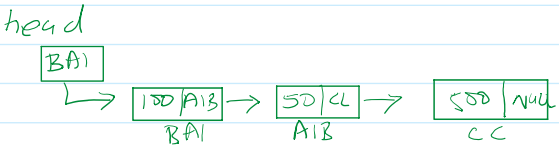
```
struct Node {

    int data;
    struct Node * next;
};
```

```
    struct Node *next;
};
```

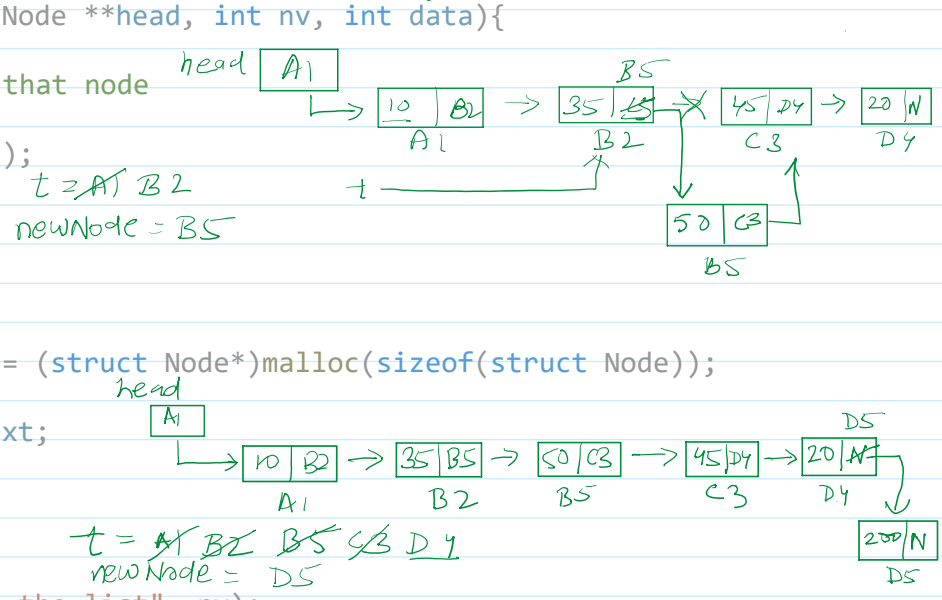
```
void addNodeAtStart(Node **head, int value){
    Node *newNode = (Node *)calloc(1, sizeof(Node));
    newNode->data = value;
    newNode->next = NULL;
    if(*head == NULL){
        *head = newNode;
        return;
    }
    newNode->next = *head;
    *head = newNode;
}
```




```
void addNodeAtEnd(struct Node **head, int value){
    struct Node*newNode = (struct Node *)calloc(1, sizeof(struct Node));
    newNode->data = value;
    newNode->next = NULL;
    if(*head == NULL){
        *head = newNode;
        return;
    }
    struct Node *t = *head;
    for(;t->next;t = t->next);
    t->next = newNode;
}
```



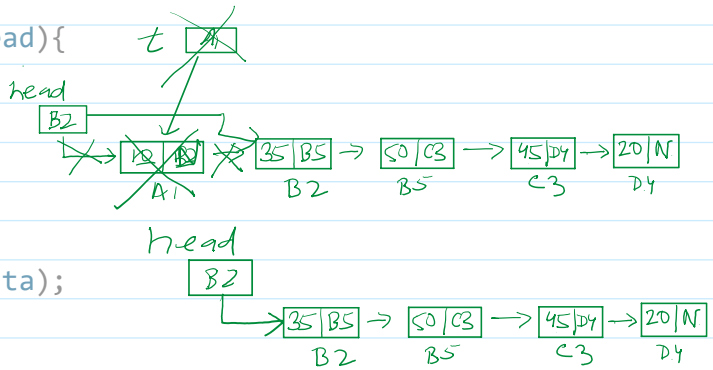

```
void insertNodeAfterANode(struct Node **head, int nv, int data){
    //first find the nv node
    //Then insert the node after that node
    if(*head == NULL){
        printf("\nList is empty!");
        return;
    }
    struct Node *t = *head;
    for(;t; t=t->next){
        if(t->data == nv){
            struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));
            newNode->data = data;
            newNode->next = t->next;
            t->next = newNode;
            return;
        }
    }
    printf("\n %d is not found in the list", nv);
}
```



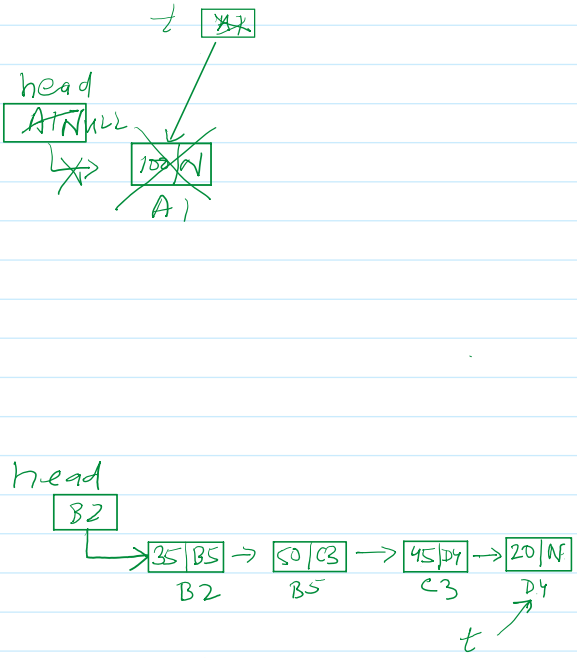
free() method is used to release memory occupied by using malloc() and

calloc() methods.

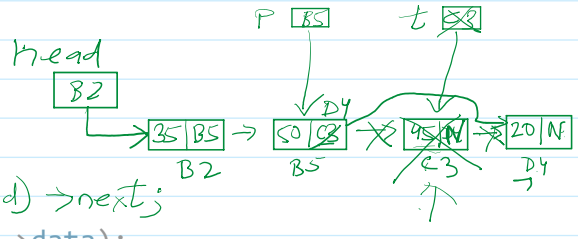
```
void deleteNodeAtHead(struct Node **head){
    if(*head == NULL){
        printf("\nList is empty!\n");
        return;
    }
    struct Node *t = head;
    printf("\nDeleted node: %d", t->data);
    *head = (*head)->next;
    t->next = NULL;
    free(t);
}
```



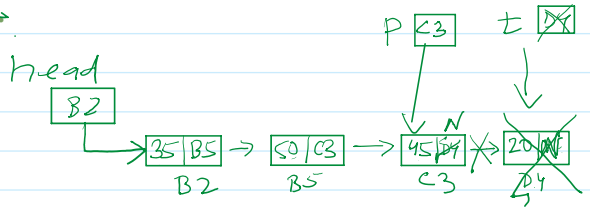

```
void deleteNodeAtTail(struct Node **head){
    if(*head == NULL){
        printf("\nList is empty!\n");
        return;
    }
    //single node
    struct Node *t = *head;
    if((*head)->next == NULL){
        *head = NULL;
        print("\nDeleted node: %d", t->data);
        free(t);
        return;
    }
```




```
void deleteANode(struct Node **head, int sv){
    if(*head == NULL){
        printf("\nList is empty!\n");
        return;
    }
    //single node
    struct Node *t = *head, *p;
    if((*head)->next == NULL){
        if(t->data == sv){
            *head = NULL;   *head = (*head)->next;
            printf("\nDeleted node: %d", t->data);
            free(t); return;
        }
        else{
            printf("%d node is not present in the list.", sv);
        }
        return; //local t will be deleted after this
```



```
        else{
            printf("%d node is not present in the list.", sv);
        }
        return; //local t will be deleted after this.
    }
    for(p = t; t; p = t, t = t->next){
        if(t->data == sv){
            printf("\nDeleted node: %d", t->data);
            p->next = t->next;
            t->next = NULL;
            free(t);
            return;
        }
    }
    printf("\n%d is not present in the list", sv);
}
```



sv = 20
t = ~~B2~~ ~~B5~~ ~~C3~~ D4
p = ~~B2~~ ~~B5~~ C3